Radar-based Sentiment Analysis and Recognition

Mohamed-anas Abbouchi[1] , **Haider Azam**[2] , **Mudassar Iqbal**[3]
[1]Aalto University
[2]Polytechnical University of Catalunya
[3]Technische Universität Braunschweig

Abstract

1 Introduction

Sentiment analysis commonly refers to the process of identifying and extracting subjective information from text, speech, or other forms of data [Ain *et al.*, 2017]. In this study, we focused on sentiment analysis using radar-based data. Radar sensors are a powerful tool for capturing subtle movements and gestures. They are non-intrusive, privacy-preserving, and much easier to deploy in real-world scenarios compared to other modalities such as cameras or wearable sensors [Sun *et al.*, 2020]. The data from these sensors have proven useful in applications such as gesture recognition [Sun *et al.*, 2020], object detection [Manjunath *et al.*, 2018] and human activity recognition [Froehlich *et al.*, 2024].

The main focus of researchers in the field of sentiment analysis has been on text-based and speech-based data, with a significant amount of work done on NLP models [Rokade *et al.*, 2025] and speech processing techniques. The objective was always to create models that understand human emotions through media but rarely through their body movements. Consequently, the use of radar-based data for sentiment analysis and recognition is still a relatively unexplored area, with limited research and few datasets available. The characteristics of radar signals, such as their ability to capture micro-movements and gestures, make them a promising source of information for micro-movement based experiments such as sentiment analysis [Sun *et al.*, 2020].

The objective of this paper is to explore the potential of radar-based data for sentiment analysis and recognition. We propose different approaches to transform radar signals into training data and evaluate how effective the classification models are in recognizing different sentiments. We split the paper into three experiments, each focusing on a different aspect of the problem. The first experiment focuses on feature-based classification using a mix of kinematic, behavioral and spatial features extracted from the radar signals. The second experiment is based on deep learning with raw sensor data. The third is [FILL HERE].

2 Related Work

This research is based on the experiments conducted by [Zuo *et al.*, 2025], in which they introduced "a multimodal radio frequency dataset" for both human behavior recognition and emotion analysis. By conducting a series of experiments on 44 participants, they collected data from 13 radars, 8 RFID tags, LoRa devices and infrared cameras in 4 different campaigns that each focused on a different aspect of human behavior. We will be interested in this paper on the third campaign that was built around 6 sentiment states: *depression, distraction, excitement, focus, relaxation and stress*.

3 Experiment 1: Feature-Based Classification

3.1 Preprocessing

The data is currently composed of a sequence of XYZ coordinates representing the position of a target in 3D space over time. The first step in preprocessing is to synchronize the data from different radar sensors to ensure that the data from different sensors is aligned in time, as this will allow for accurate feature extraction. Once the data is synchronized, the data is normalized on a user basis so the user-specific biases are removed.

With the data synchronized and normalized, we can proceed to segment the data into smaller windows. This is done to capture the temporal dynamics of the target's movement. The window size and stride are hyperparameters that can be tuned based on the specific application and dataset. In this study, we experimented with different window sizes and strides to find the optimal configuration for our classification task. Lower window sizes tend to underperform and higher window sizes tend to overfit the data and lose generalization. The optimal window size was found to be 90 frames with a stride of 20 frames. This configuration allows for capturing the temporal dynamics of the target's movement while also providing enough data for training the classification models.

3.2 Feature Extraction

Given the preprocessed data, we can extract a set of features that capture the kinematic, behavioral and spatial characteristics of the target's movement. The features are extracted from each window of data and are used as input to the classification models. The features are divided into three categories: motion dynamics, statistical and directional features.

Feature	Description
Path Efficiency	Ratio of displacement to trajectory length
Mean Speed	Average speed.
Std. Speed	Variability in speed.
Median Speed	Median speed.
25th Percentile Speed	Lower quartile of speed.
75th Percentile Speed	Upper quartile of speed.
Mean Acceleration	Average rate of change in speed.
Std. Acceleration	Variability of acceleration.
Median Acceleration	Median acceleration.
25th Percentile Acceleration	Lower quartile of acceleration values.
75th Percentile Acceleration	Upper quartile of acceleration values.

Table 1: Motion dynamics features.

Feature	Description
Mean Jerk	Average change of acceleration
Std. Jerk	Variability in jerk values.
Median Jerk	Median jerk.
25th Percentile Jerk	Lower quartile of jerk values.
75th Percentile Jerk	Upper quartile of jerk values.
Spectral Centroid	Center of mass of the movement frequency spectrum.
Spectral Entropy	Complexity of the frequency spectrum.
Speed CV	Normalized measure of speed variability.
Direction Change Rate	Rate of changes in movement direction.
Speed Autocorrelation	Temporal dependence of cursor speed.
Tortuosity	Degree of path curvature relative to its overall length.

Table 2: Statistical and directional features.

The motion dynamics features capture the speed, acceleration and jerk of the target's movement. The statistical features capture the distribution of the target's movement in terms of speed, acceleration and jerk. The directional features capture the directionality of the target's movement in terms of turn angles and explained variance ratios. The extracted features are summarized in Tables 1, 2 and 3.

3.3 Training Loop

With the current setup, the data is now composed of 27 features per radar sensor. Given that we have 13 radar sensors, the total number of features is 351 over more than 600 windows. For an effective training of the classification models, we need to reduce the dimensionality of the data and select the most relevant features. To find the optimal configuration of window size, stride and number of features, we implemented a training loop that iterates over different configurations and evaluates the performance of the classification models using Leave-One-Subject-Out (LOSO) cross-validation. It uses the ANOVA F-test to select the top-k features for each radar sensor and concatenates them to form the final feature set. The classification models are then trained on the training set and evaluated on the test set. The accuracy and weighted F1-score are recorded for each configuration and the best configuration is selected based on the highest mean accuracy across all folds. The training loop is summarized in Algorithm 1.

Feature	Description
Speed Skewness	Asymmetry of the speed distribution.
Fraction Stationary Time	Proportion the target remains stationary.
Mean Turn Angle	Average angle between consecutive movement segments.
Turn Angle Entropy	Diversity and unpredictability of turning angles.
Explained Variance Ratio 1	Variance explained by the first principal component of the trajectory.
Explained Variance Ratio 2	Variance explained by the second principal component.
Explained Variance Ratio 3	Variance explained by the third principal component.

Table 3: Statistical and directional features.

Algorithm 1 Feature Selection and LOSO Classification

for each window configuration (w, s) **do**
  for each feature count $k \in K$ **do**
    for each Leave-One-Subject-Out split **do**
      Split the data into training and testing sets.
      for each radar $r = 1, \ldots, 13$ **do**
        Select the top-k features using the ANOVA F-test.
        Transform the training and testing feature sets.
      end for
      Concatenate the selected features from all radars.
      for each classifier $m \in \mathcal{M}$ **do**
        Train classifier m.
        Evaluate on the test set.
        Record the accuracy and weighted F1-score.
      end for
    end for
    Compute the mean accuracy and weighted F1-score across all folds.
  end for
end for
for each classifier **do**
  Select the configuration with the highest mean accuracy.
end for
Output: Best window size, stride, feature count, accuracy, and weighted F1-score for each classifier.

3.4 Results

The best configuration for each model is selected based on the highest mean f1-score across all folds. As previously mentioned, the most optimal configuration across all models was found to be a window size of 90 frames, a stride of 20. The

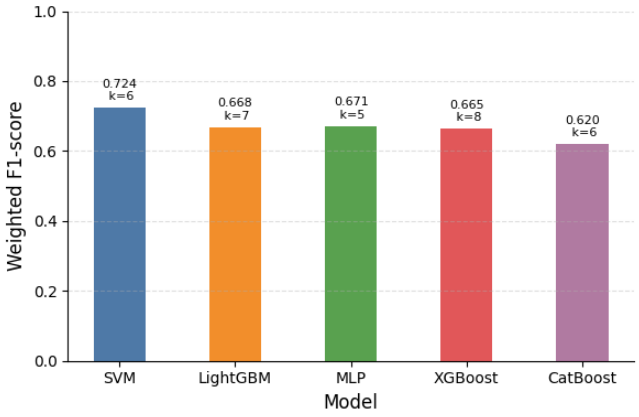


Figure 1: Models comparison based on F1 score.

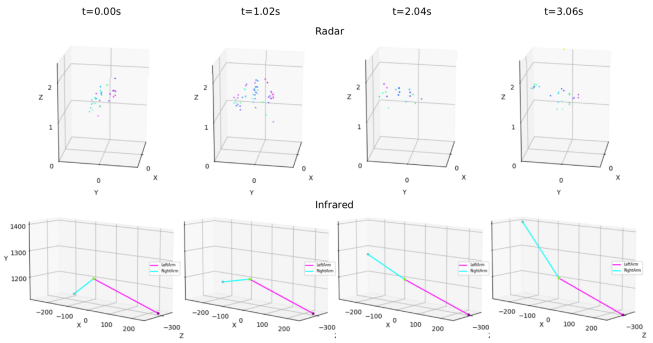


Figure 2: Correspondence between radar and infrared frames. Each color in the radar scatter plot corresponds to one of the radar sensors. The shape of the right arm and right side of the torso is roughly discernible in the animation of the radar data.

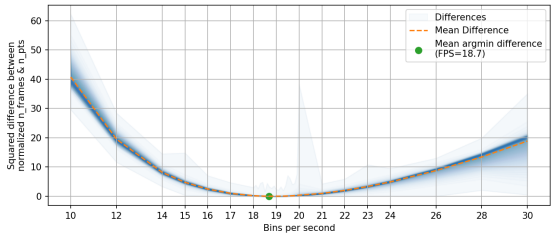


Figure 3: Optimizing the frame rate for radar data.

results of the training loop in Figure 1 are based on models trained with this configuration.

It shows that the SVM model outperforms the other models with a mean f1-score of 0.72, followed by LightGBM, xGBoost and MLP achieving mean f1-scores in the range of 0.66 and 0.67. The CatBoost classifier underperforms with a mean f1-score of 0.62.

4 Experiment 2: Deep Learning with Raw Sensor Data

4.1 Dataset

For the scope of this experiment, we will utilize the radio and infrared modalities of the RF-Behavior dataset. We will investigate the first 3 campaigns of the dataset which are 21 motions, 10 activities, and 6 emotions respectively. Within each behavior class, there are multiple recordings of different humans (one person per recording) and for each human, and in the first two campaigns, there are multiple repetitions of the same behavior (mean duration: 3.73 seconds), whereas in the third campaign, just one long recording of the behavior exists (mean duration: 152.29 seconds). Furthermore, the first two activities (i.e. walking and running) can also be considered long recordings of the same behavior because only a few steps need to be observed to recognize walking or running but these recordings have a mean duration of 14.3 seconds. So, for the purpose of this experiment, these two activities and all 6 emotions were cut into random sections of 4-6 seconds to roughly match the duration of all other behaviors. We expect it to increase diversity within that behavior and break long duration patterns (e.g. the specific path taken) not associated with that behavior's definition.

4.2 Preprocessing: Uniform Batches

Sampling Rate

The radar data is collected from 13 sensors mounted on the ceiling and the floor. Each sensor detects an arbitrary number of 4D points (density, x,y,z) at arbitrary timestamps. To make the input uniform (batch, time, points, coordinates), the data from all sensors is distributed into evenly spaced time bins. The frame rate is chosen in a way that the number of frames in a behavior recording equals the mean number of points in one frame. This frame rate is empirically calculated by binning each recording from the dataset on all frame rate values ranging from 10 to 30 with 0.1 step size. For a given recording, after binning it on one FPS value, the total number of bins is noted and the mean value of the number of points in each bin is calculated. These two values are collected in two separate arrays called n_frames and mean_n_points for all FPS values. These arrays are then each normalized by subtracting their respective mean and dividing by their respective standard deviation. Then element wise squared difference is calculated between the corresponding values in the arrays. For each FPS value, the corresponding squared difference is averaged across all behavior recordings. This results in the U shaped plot shown in Figure 3. The minimum point occurs at frame rate 18.7 (i.e. time bin of around 53.5ms). At this frame rate, an average behavior recording from this dataset would have the same number of frames as the number of 4D points in an average frame. (i.e. the time and points dimension in the final tensor would be of similar size).

The infrared data was sampled at a fixed rate of 100 Hz and consists of 3 or 6 7D landmarks per frame with each landmark consisting of 3 position coordinates and 4 orientation quaternion. For scaled dot product attention, the compute cost scales quadratically with sequence length but for the scope of this experiment, we do not perform any preprocessing on the infrared sampling rate.

Jagged Input Tensors

The duration of a behavior recording can be variable within a behavior class and across classes. So the tensor representation of sensor data from any modality will jagged across time. we handle this by padding the shorter sequences on the right side (larger indices) of the input tensor by copies of the last frame of the recording. During this experiment, we did not explore the application of other padding strategies.

For both radar and infrared modalities, the number of points within a frame depends firstly on the number of sensors in use. In this dataset, there are 13 radar sensors but not all recordings contain data from all of the sensors. Some radar sensors also may not provide any input due to high distance. The infrared data only has 3 upper body landmarks in the first campaign and the following campaigns include 3 more landmarks on the lower body. So, if data from two different campaigns is used in training, a variability in the points dimension of infrared data has to be managed. Furthermore, in downstream applications, the sensor setup may be totally different from the campaigns of RF-Behavior. Secondly, each radar sensor can return a variable number of points in any time bin. To manage the variability in the points dimension, we split a each batch of behavior *recordings* into numerous mini batches of *frames* each have the same number of points. Then we process each pseudo batch separately and then reorganize and concatenate the output tensors.

4.3 Model

Baseline Architecture: Transformers

Since the time and points dimensions are of variable size, we propose using two stacks of transformers to collapse each of them to size 1.

For the points dimension, we implement a Point Cloud Encoder consisting of a linear projection layer followed by a BERT [Devlin *et al.*, 2019] encoder implementation from the HuggingFace library. It treats a collection of points from a single frame as a sequence of tokens and applies self attention to learn the relationships between them which is then pooled into a single embedding vector per frame. For radar, the order of the points contains no meaning so we do not use any positional encoding. For infrared, there is still no information in the exact sequence of the points so sinusodial or rotary positional encoding are not used. However, learned positional embeddings can be added to the projection of the landmarks to help the model identify which body part is at what coordinates. The point cloud encoder takes an input of the shape (batch, points, coordinates) and outputs a tensor of the shape (batch, hidden_size).

For the time dimension, we implement Temporal Encoder consisting of a linear projection layer followed by a Llama [Touvron *et al.*, 2023] decoder borrowed from the HuggingFace library. The time dimension obviously contains sequential information and llama models use rotary positional embeddings and causal attention to learn the relationships between input tokens (frames in our case). The output of llama model is another sequence of vectors so we apply pooling again to obtain a single embedding vector for the entire recording. The padding added to the time dimension during preprocessing is handled by an attention mask that marks the real frames with 1s and the padding frames with 0s. Our temporal encoder expects an input of the shape (batch, time, hidden_size) and outputs a tensor of the shape (batch, hidden_size).

Since infrared data has a high sampling rate and the landmarks maintain their order in the input (e.g. left shoulder always appears at the first index), we also define an optional 1D convolution based downsampling block that scans across the time dimension with a stride of 2 and reduces the number of frames by half. The channel dimension can be all coordinates of all landmarks from a single frame flattened into a single vector. This restricts the model architecture to a fixed number of points. Alternatively, the channel dimension can be the coordinates of a single landmark and the points dimension can be transposed and merged into the batch dimension (to be undone later). This way the same weights are applied to all landmarks and the model is also free to handle a variable number of landmarks.

The final sequence of model components is as follows: [Optional Downsampler] $\rightarrow$ Point Cloud Encoder $\rightarrow$ Temporal Encoder $\rightarrow$ Output Projection layer.

Hyperparameter	Values
# Transformer blocks	1-6
# Convolutional Blocks	0-4
Positional Embeddings for BERT/Infrared	True/False
Pooling Strategy	Mean, CLS, EOS
Attention Heads	2^i (i = 2, ..., 5)
Embedding Size	2^i (i = 4, ..., 10)
Conv Channels	2^i (i = 4, ..., 7)

Table 4: Architectural hyperparameters.

Architectural Hyperparameters

The transformer architecture is highly configurable and can be scaled in multiple dimensions. We define a list of possible values for each hyper parameter and then sample 10,000 random combinations and measure the total number of parameters in the resulting model. We then distribute the models into 20 geometrically spaced bins ranging from 70k to 100M parameters. We then sequentially process one random model from each bin and evaluate its performance. See Table 4 for possible values for each hyperparameter.

Supervised Classification

To compare the sampled models and optimize the hyperparameters, we define a simple supervised classification task for classifying complete behavior recordings among all classes in a campaign. The campaigns 1-3 involve 21, 10, and 6 classes respectively. We train a separate classifier for each campaign. The dataset is split into train, validation and test sets using the user_ids to avoid any personal style or patterns of a given user being leaked to the model during training. We place 4 users in the test set and 2 users in the validation set with user having performed 8 repetitions. Rest of the recordings are used for training (160 recordings per class). We train the model using cross entropy loss and use an early stopping strategy with a patience of 5 epochs. The model is trained for a maximum

of 100 epochs with a batch size of 32 and a learning rate of 1e-3 with the AdamW optimizer. The model with the lowest validation loss is saved and evaluated on the test set using weighted classification F1 score.

Unsupervised Contrastive Learning

Downstream applications of a model that classifies radar or infrared recordings among the classes of RF-Behavior seem limited. Just as the generalized input capability of our model, we explore the possibility of having a generalized output capability as well. Typically, the classification head of a model can be removed to use it as a feature extractor for transfer learning or fine-tuning. But still, training for rigid labels can prime the model to ignore many other latent features it could have been able to extract. Zero-shot models like CLIP [Radford *et al.*, 2021] have shown that training an embedding model can be more beneficial downstream.

We select the best model architectures from our grid search results and set the output size to 256. We train these models in a semi supervised fashion using the NT-Xent loss introduced in the SimCLR framework [Chen *et al.*, 2020] to increase the cosine similarity between embedding vectors of different instances of the same class and decrease their similarity with instances of other classes in the batch. We split the dataset by placing two behaviors in validation and two behaviors in test. For this model, we limit ourself to campaign 1 and 2 and train separate models for each campaign. We then evaluate these embeddings in a 1 shot fashion on the remaining 2 unseen classes. We also evaluate how the performance of a supervised classification head scales with the amount of samples used for training that head. For example, in campaign 1, the embedding model is trained on 17 classes and the classification head is trained to classify the untouched embeddings among the 2 behaviors from the original test set. For training the classification head, we split the data based on user_id.

One of the modalities will have a lower performance than the other and in order to enable the model for that modality to learn from all other data that we have available, we align the embedding space of all modalities and train both models jointly. Since the sensor data from different modalities was collected simultaneously and radar and infrared data are syncronized, we can use a symmetric contrastive loss (like CLIP [Radford *et al.*, 2021]) to align the two modalities. We map data from all sensor types onto embedding vectors of the same size and use cross entropy to increase the similarity between embeddings coming from different sensors but of the exact same recording instance and decrease the similarity with any other instance of the same class.

4.4 Results

Through our grid search, we discovered that using CLS pooling in the point cloud encoder collapses the model's representation and all inputs are mapped to almost the same vector by the model. We also measured that including more convolutional blocks in the infrared model significantly reduces the model inference and convergence time during training.

We evaluate our supervised models on weighted (balanced) classification F1 score and the results for the best models are shown in Table 5 and the distribution of all our trained models is illustrated via a scatter plot in Figure 4.

Campaign	# Classes	Modality	# Parameters	Test F1
C1	21	Radar	1,311,125	94.41%
C1	21	Infrared	526,869	96.56%
C2	10	Radar	106,826	91.97%
C2	10	Infrared	1,541,850	98.12%
C3	6	Radar	1,349,766	46.12%
C3	6	Infrared	243,798	56.16%

Table 5: Supervised Classification Results.

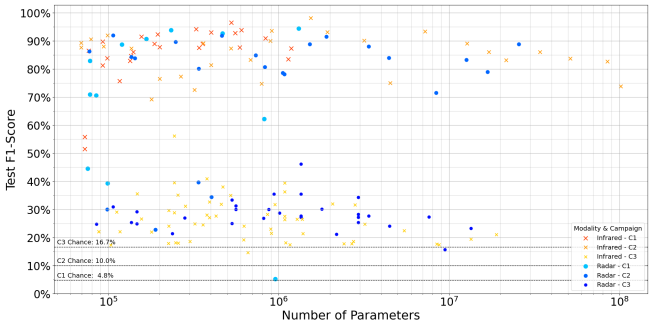


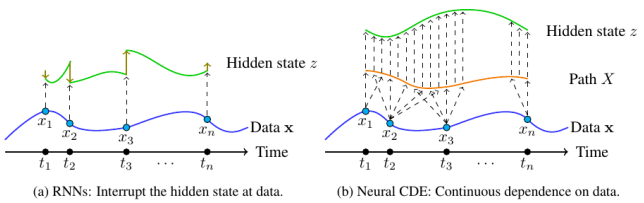
Figure 4: Scatter plot of F1-scores for different classification models.

For C3, in addition to grid search, we also employed genetic search to crossover and mutate the hyperparameters of two successful models to create a new model in an attempt to maximize the classification F1 score but no new model out performed the existing ones. We also tested various sample durations up to 36 seconds for the emotions but we did not observe any significant improvements. We expect this could be due to subtle nature of the behaviors in this campaign and we might need more landmarks or sensors to capture the micro-movements associated with these behaviors. We also observed that the radar modality consistently underperformed compared to infrared modality across all campaigns, which could be attributed to the lower resolution, sparse sampling and higher noise levels in radar data.

[NOTE: THE RESULTS OF UNSUPERVISED LEARNING WILL BE ADDED HERE. All the unsupervised experiments done so far, used CLS pooling in the point cloud encoder and therefore the results were not much better than random chance. Now after having completed thegrid-search, we will re-run the unsupervised experiments with the best architectures and report the results ASAP.]

5 Experiment 3: Neural Network-Based Interpolation and Classification

The radar-based dataset consists of 13 separate radar sensors each recording data at different timestamps, in order to unify the sensors to a single timeline, interpolation is required. This method consists of performing interpolation using Neural Networks and comparing with classical methods, emotion classification is also performed.



(a) RNNs: Interrupt the hidden state at data. (b) Neural CDE: Continuous dependence on data.

Figure 5: Comparison of hidden state updates between RNN and NCDE.

5.1 Preprocessing

Using the preprocessing sample code provided in the dataset, the radar sensors' data were converted to 39 XYZ coordinates, afterwards, the timestamps were rounded to 1 decimal place and the sensors concatenated together into a single csv file for each user for each emotion. These were then concatenated together to get a singular csv file containing the processed data of every user, emotion and sensor. Grouping by user_id and emotion, a sparse sequence of observations from all the sensors combined can be retrieved.

To avoid any overlap between dataset splits and to test on unseen users, the user_ids were split into train, valid and test sets with ratios 0.6/0.2/0.2 . Normalization is performed by setting mean to 0 to allow the model to only train on the residual. Min-max normalization is applied to the timestamps to set the minimum to 0 and maximum to 1.

While training the models, 0.1 to 0.3 of the sequence is randomly set aside as the target sequence, this is set to 0.2 for validation and testing with a static random state for reproducibility. An observation mask containing the non-Nan values is obtained for both the input and target sequence, this is combined with the input sequence to allow the model to focus on observations and is used to train the model using only non-Nan target values. Last Observation Carried Forward (LOCF) is then applied on the data to counter-act the sparsity of the dataset.

5.2 Model Selection

The proposed method consists of training a Neural Controlled Differential Equation (NCDE) model by Kidger et. al [Kidger *et al.*, 2020] for interpolation and classification using the torchcde library. It was selected due to it's ability to deal with irregular timestamps. As shown in Figure. 5, the main advantage over Recurrent Neural Networks (RNNs) is the continuous update of it's hidden state, this is achieved by solving and integrating the differential equation in Equation. 1 where $f(t, z(t))$ and $g(x_0)$ are Neual Networks (NNs) and X is a Hermite cubic splines interpolation of sparse input sequence.

$$\frac{dz(t)}{dt} = f(t, z(t))\frac{dX(t)}{dt}, \qquad z(t_0) = g(x_0), \qquad (1)$$

Shown in Figure. 6 is the workflow of the proposed method, it starts with interpolating the input sequence using a differentiable hermite cubic spline to obtain a continuous control data X, whose first instance x(0) is passed through an MLP to get z(0) and both are fed into the cdeint function along with the target timestamps dictating at what time we want the hidden states z(t), the output sequence is fed into an MLP decoder to get the predicted values while the final hidden state is fed into an MLP classifier to get emotion probabilities.

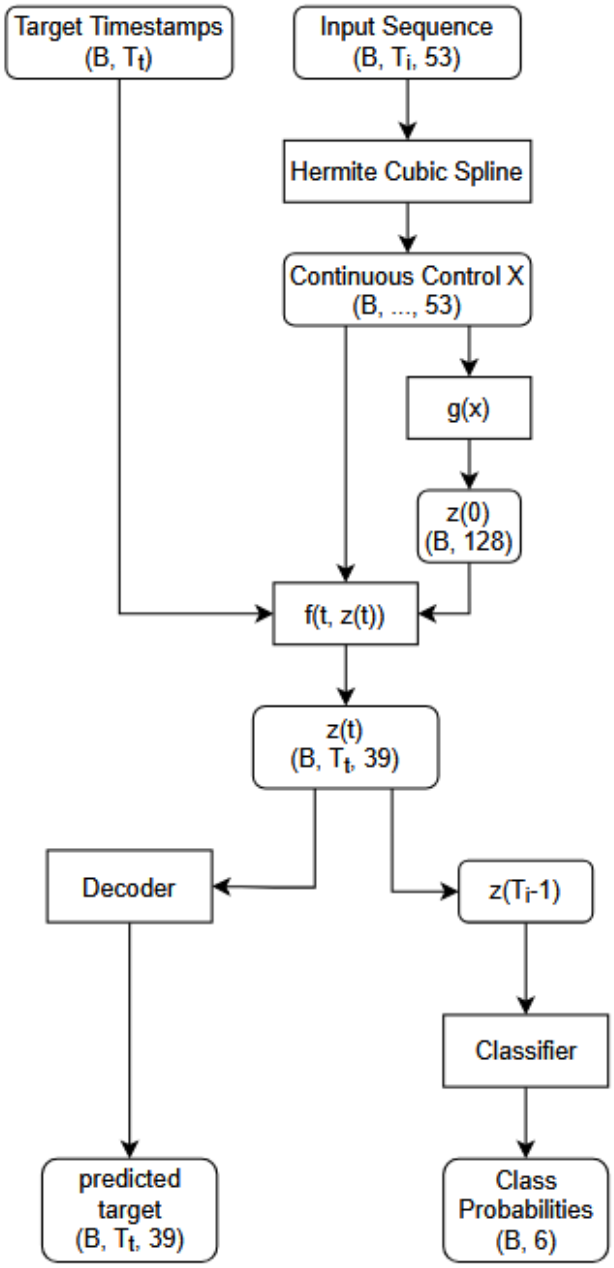


Figure 6: Workflow of the proposed method, it uses a differentiable hermite cubic spline which can be integrated to obtain the hidden state at arbitrary timestamps, the final hidden state is used to obtain class probabilities similar to RNNs.

5.3 Metrics

The model was trained using a combination of Mean Squared Error (MSE) loss for the sensor interpolation and Cross-

Hyperparameter	Value
Hidden channels	128
Batch size	1
Gradient Accumulation	8
Learning rate	1×10^{-4}
Optimizer	AdamW
Number of epochs	20
Gradient clipping max norm	1.0
ODE solver	RK4

Table 6: Hyperparameters used for training the NCDE model.

Method	MSE	Cross-Entropy	Accuracy
NCDE model (final epoch)	0.0992	1.7809	30.55
NCDE model (best classify loss)	0.0992	1.7809	30.55
NCDE model (best MSE loss)	0.0999	1.7913	22.22

Table 7: NCDE model performance on test set.

Entropy loss for the emotion classification. Accuracy was used to measure the classification performance of the model on the test set.

5.4 Hyperparameters

Table. 6 shows the hyper-parameters, batch size was set to 1 as the length of the target sequences varied, to counteract this, gradient accumulation every 8 steps was implemented to mimic the effect of having a batch size of 8. Gradient clipping was implemented to avoid exploding gradients,

5.5 Results

The model was trained for 20 epochs and the weights with the best MSE loss and cross-entropy loss on valid set were saved along with final epoch weights. The performance of the model on the test set is listed in Table. 7. Despite the small MSE loss, the model still performs poorly when interpolating, this is due to the mean of the input sequence being 0, leading to a very small loss and the model output being close to 0. The same effect can be gotten by just taking the mean of the input sequence as the predicted value.

Further comparison between regular cubic spline and NCDE is required to determine if the model is underfitting or if the input sequences does not have enough discriminatory information.

6 Discussion

Acknowledgments

References

[Ain *et al.*, 2017] Qurat Tul Ain, Mubashir Ali, Amna Riaz, Amna Noureen, Muhammad Kamran, Babar Hayat, and A-u Rehman. Sentiment analysis using deep learning techniques: a review. *International Journal of Advanced Computer Science and Applications*, 8(6), 2017.

[Chen *et al.*, 2020] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607. PmLR, 2020.

[Devlin *et al.*, 2019] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[Froehlich *et al.*, 2024] Ann-Christine Froehlich, Desar Mejdani, Lukas Engel, Johanna Braeunig, Christoph Kammel, Martin Vossiek, and Ingrid Ullmann. A millimeter-wave mimo radar network for human activity recognition and fall detection. In *2024 IEEE Radar Conference (RadarConf24)*, pages 1–5, 2024.

[Kidger *et al.*, 2020] Patrick Kidger, James Morrill, James Foster, and Terry Lyons. Neural Controlled Differential Equations for Irregular Time Series. *Advances in Neural Information Processing Systems*, 2020.

[Manjunath *et al.*, 2018] Ankith Manjunath, Ying Liu, Bernardo Henriques, and Armin Engstle. Radar based object detection and tracking for autonomous driving. In *2018 IEEE MTT-S International Conference on Microwaves for Intelligent Mobility (ICMIM)*, pages 1–4, 2018.

[Radford *et al.*, 2021] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.

[Rokade *et al.*, 2025] Gayatri Rokade, Radhe Ughade, and Palash Gaurshettiwar. Deep learning for sentiment analysis. In *2025 4th International Conference on Sentiment Analysis and Deep Learning (ICSADL)*, pages 240–244, 2025.

[Sun *et al.*, 2020] Yuliang Sun, Tai Fei, Xibo Li, Alexander Warnecke, Ernst Warsitz, and Nils Pohl. Real-time radar-based gesture detection and recognition built in an edge-computing platform. *IEEE Sensors Journal*, 20(18):10706–10716, 2020.

[Touvron *et al.*, 2023] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[Zuo *et al.*, 2025] Si Zuo, Yuqing Song, Sahar Golipoor, Ying Liu, Xujun Ma, and Stephan Sigg. Rf-behavior: A

multimodal radio-frequency dataset for human behavior and emotion analysis, 2025.